Clustering incrémental binaire
Invariant par translation et mémorisation de récence

1. Contexte et objectifs

Nous proposons un algorithme de clustering incrémental pour des instances codées en binaire de longueur fixe L. Trois extensions majeures sont intégrées :

1. *Invariance par translation*,
2. Opérations *purement binaires*,
3. Mémorisation *d'instances récentes* pour rattachement direct.

2. Invariance par translation

Chaque cluster C est défini par :

$$\begin{cases} \text{base } b \in \{0,1\}^L, \\ \text{offsets } \{d_i\} \quad \text{tels que } x_i = b \ + \ d_i. \end{cases}$$

Pour une nouvelle instance x, on calcule

$$d = x - b \quad \text{(soustraction binaire bit-à-bit)}.$$

Si $d < 0$, on met à jour la base en $b' = x$ et tous les offsets $d_i \leftarrow d_i + (b - b')$. Ainsi, un décalage commun c appliqué à toutes les instances ne modifie que la base, pas les offsets.

3. Critère MDL binaire

Pour chaque cluster de taille n et base b, on définit

$$b_{\max} \ = \ \max_i \lfloor \log_2(d_i) \rfloor + 1,$$

le nombre de bits nécessaires pour coder les plus grands offsets. Le *coût MDL* du cluster vaut :

$$L_{\text{modèle}} = L \quad \text{et} \quad L_{\text{données}} = n \times b_{\max}.$$

À l'arrivée d'une instance x, on compare :

$$\Delta L = \big[L + (n+1)\, b'_{\max}\big] - \big[L + n\, b_{\max} + L_{\text{création}}\big],$$

avec $L_{\text{création}} = L$ pour créer un nouveau cluster. On retient l'option de plus faible ΔL.

4. Mémorisation d'instances récentes

On maintient un *buffer* FIFO $\mathcal{R}$ des R dernières instances vues, chacune associée à son cluster. Soit $\mathcal{R} = \{(x_{t-1}, c_{t-1}), \ldots, (x_{t-R}, c_{t-R})\}$. Lors de l'arrivée de x_t, on cherche dans $\mathcal{R}$ un x_r tel que

$$\text{Hamming}(x_t, x_r) \;\leq\; \tau,$$

pour un seuil $\tau \geq 0$.

— Si un tel x_r existe, on *rattache directement* x_t au cluster c_r.
— Sinon, on effectue la procédure MDL décrite en section 3.

Après attribution, on met à jour :

$$\mathcal{R} \;\leftarrow\; \big\{(x_t, c_t)\big\} \;\cup\; \big\{\text{les } R-1 \text{ premiers de } \mathcal{R}\big\}.$$

Ce mécanisme assure :

— *Rattachement instantané* des valeurs déjà vues ou très proches,
— meilleure *réactivité* et *stabilité* sur les motifs récents,
— diminution du coût de calcul MDL si la donnée revient.

5. Algorithme incrémental résumé

1. Initialiser liste de clusters $\emptyset$ et buffer $\mathcal{R} = \emptyset$.
2. Pour chaque nouvelle instance x_t :
   (a) Si $\exists\, (x_r, c_r) \in \mathcal{R}$ avec $\text{Hamming}(x_t, x_r) \leq \tau$:

   $$\text{assigner } x_t \to c_r.$$

   (b) Sinon, pour chaque cluster C_j, calculer ΔL_j par MDL (section 3), et créer un nouveau cluster coûte L bits. Choisir l'option minimale et mettre à jour base/offsets.
   (c) Mettre à jour $\mathcal{R} \leftarrow \{(x_t, c_t)\} \cup \mathcal{R}[1 : R-1]$.
3. Fin.

6. Conclusion

Cette extension permet de :

— reconnaître immédiatement les réapparitions d'instances ou les petites variations ;

— combiner MDL et *mémoire courte* pour un compromis stabilité/plasticité ;

— garantir l'invariance par translation et conserver une implémentation *purement binaire*.